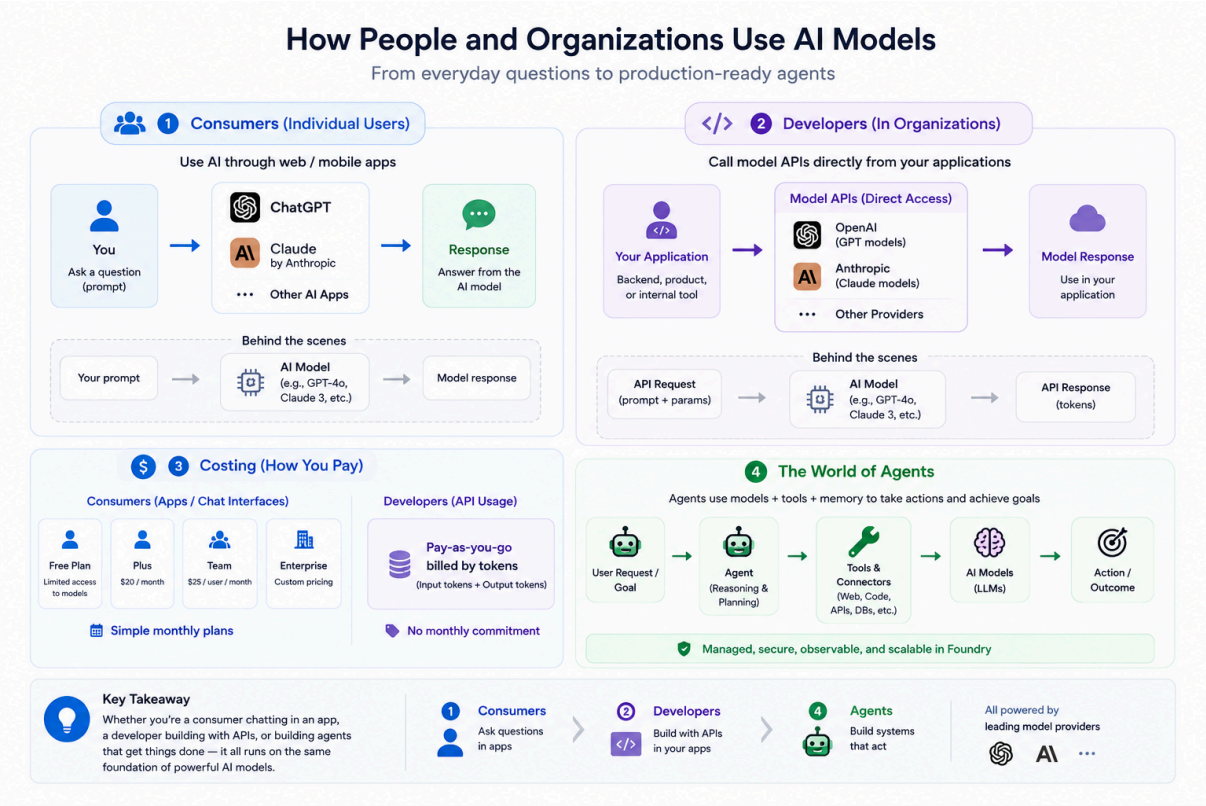
Building the Foundation: Models, Tools, and the Developer Workflow

How People and Organizations use AI Models





At a high level, there are different ways people interact with AI models.

For individual users, the experience is usually very simple. You open an app such as ChatGPT, Claude, or another AI assistant. You type a question, upload a file, or ask for help with a task. Behind the scenes, that app sends your prompt to an AI model, the model generates a response, and the app shows that response back to you in a friendly chat interface.

So as a consumer, you normally do not think about APIs, tokens, endpoints, or infrastructure. You just use the application.

Now compare that with developers inside an organization. Developers usually want to bring AI into their own applications, products, or internal tools. In that case, instead of using a chat interface directly, the application calls a model API. The application sends the prompt, along with parameters such as the model name, temperature, or other settings. The model then returns a response, and the developer decides how that response should be used inside the application.

This is important because many business AI solutions are not just people chatting with an AI assistant. They are applications that use AI models behind the scenes.

For example, a customer support portal might use an AI model to summarize tickets. A sales application might generate email drafts. A document processing system might extract information from uploaded files. In all of these cases, the model is being used through an API.

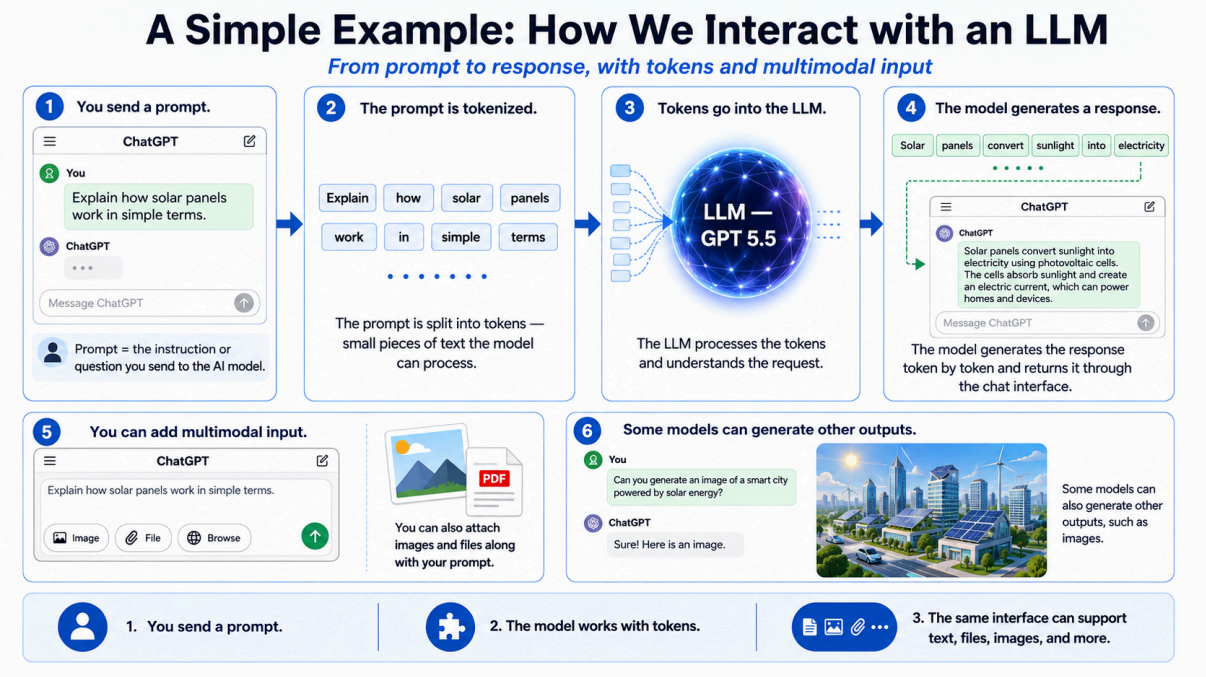
The image also shows the costing side. For consumers, pricing is usually simple. You may have a free plan, a monthly plan, a team plan, or an enterprise plan. But for developers, pricing is often based on usage. That usage is commonly measured in tokens. Input tokens are the text sent to the model, and output tokens are the text generated by the model.

This is why tokens matter. They affect both how the model processes language and how much the solution costs.

An agent goes beyond a simple prompt-and-response interaction. An agent can use a model, but it can also use tools, connectors, memory, and instructions to complete a goal.

So instead of only answering a question, an agent can reason about the task, decide which tool to use, call an API, search documents,

retrieve data, and then provide a final answer or take an action. This is the direction many enterprise AI solutions are moving toward. We are not only building chatbots. We are building AI-powered systems that can work with business data and business tools.





The process starts with a prompt. A prompt is simply the instruction or question that we send to the AI model. For example, we might ask, “Explain how solar panels work in simple terms.”

Once we send that prompt, the model does not process the sentence exactly the way we see it as humans. Instead, the text is split into tokens. Tokens are small pieces of text that the model can process. A token might be a word, part of a word, punctuation, or spacing, depending on the text.

This is a key concept because large language models work with tokens. The model reads the input tokens, processes the request, and then generates output tokens one after another to form the final response.

So when the model answers the solar panel question, it is not copying from a fixed script. It is predicting and generating the response step by step, based on the prompt, its training, and any additional context provided to it.

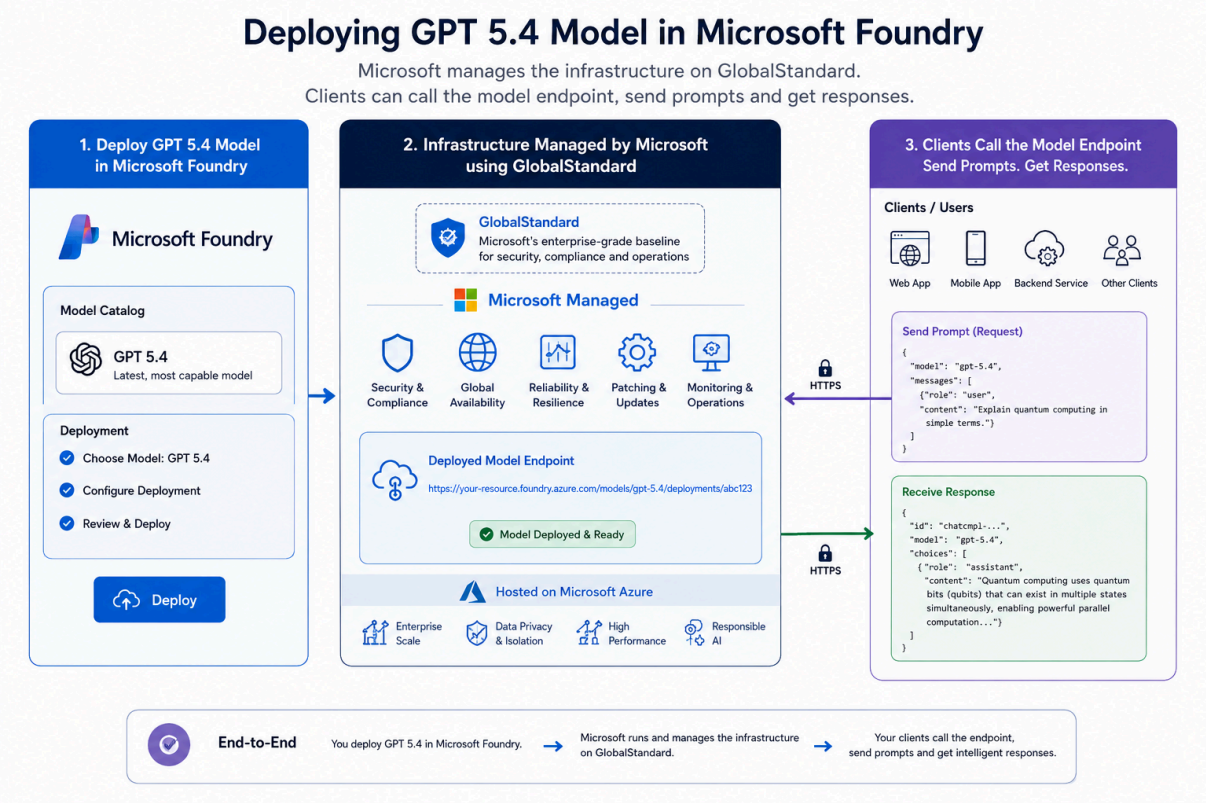
The image also highlights multimodal input. Modern AI systems are not limited to plain text. Depending on the model and the application, we can provide images, files, audio, or other types of input along with the text prompt.

For example, a user might upload a PDF and ask the model to summarize it. Or they might upload an image and ask the model to describe what it sees. This is very important for real business scenarios because enterprise data is rarely only in plain text. It can be in documents, screenshots, invoices, diagrams, forms, images, and audio recordings.

Some models can generate images, audio, or other media. So the same AI experience can support many types of input and output.

The key takeaway from both images is this: whether we are using a chat app, building an AI-powered application, or creating an agent, the foundation is still the same. We send input to a model, the model processes that input as tokens, and then it generates a response.

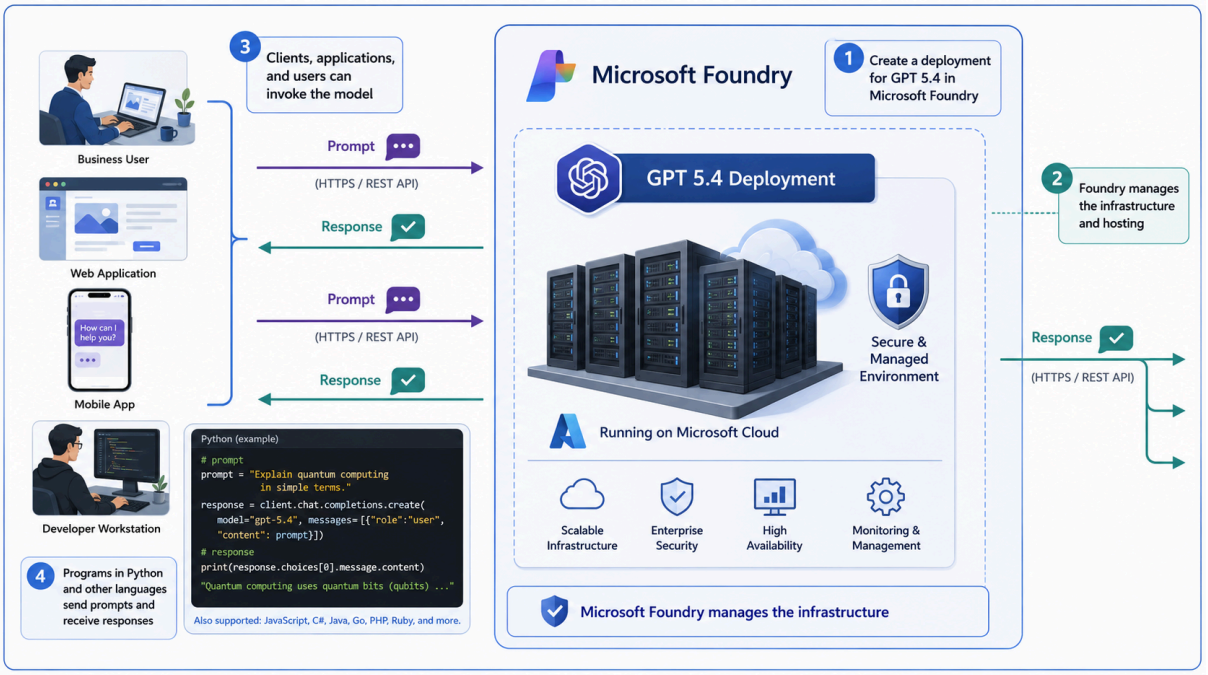
Lab - Deploying Models in Microsoft Foundry


Deploying GPT 5.4 Model in Microsoft Foundry
Microsoft manages the infrastructure on GlobalStandard.
Clients can call the model endpoint, send prompts and get responses.
1. Deploy GPT 5.4 Model in Microsoft Foundry
Microsoft Foundry
Model Catalog
GPT 5.4
Latest, most capable model
Deployment
Choose Model: GPT 5.4
Configure Deployment
Review & Deploy
Deploy
2. Infrastructure Managed by Microsoft using GlobalStandard
GlobalStandard
Microsoft's enterprise-grade baseline for security, compliance and operations
Microsoft Managed
Security & Compliance
Global Availability
Reliability & Resilience
Patching & Updates
Monitoring & Operations
Deployed Model Endpoint
https://your-resource.foundry.azure.com/models/gpt-5.4/deployments/abc123
Model Deployed & Ready
Hosted on Microsoft Azure
Enterprise Scale
Data Privacy & Isolation
High Performance
Responsible AI
HTTPS
HTTPS
3. Clients Call the Model Endpoint Send Prompts. Get Responses.
Clients / Users
Web App
Mobile App
Backend Service
Other Clients
Send Prompt (Request)
{
"model": "gpt-5.4",
"messages": [
{"role": "user",
"content": "Explain quantum computing in simple terms."}
]
}
Receive Response
{
"id": "chatcmpl-...",
"model": "gpt-5.4",
"choices": [
{"role": "assistant",
"content": "Quantum computing uses quantum bits (qubits) that can exist in multiple states simultaneously, enabling powerful parallel computation..."}
]
}
End-to-End
You deploy GPT 5.4 in Microsoft Foundry.
Microsoft runs and manages the infrastructure on GlobalStandard.
Your clients call the endpoint, send prompts and get intelligent responses.

In this lab summary, we are looking at what happens when we deploy a model, such as GPT 5.4, in Microsoft Foundry.
The first step is the deployment step. In Microsoft Foundry, we go to the model catalog, choose the model we want to use, and create a deployment for that model. The deployment is what gives us a usable endpoint. Without a deployment, the model is only available in the catalog; our applications still cannot call it directly.
Once the model is deployed, Microsoft manages the underlying infrastructure. This is an important point for students to remember. We are not provisioning virtual machines, installing model runtimes, or managing GPUs ourselves. With a deployment type such as Global Standard, Microsoft handles the model hosting infrastructure, availability, scaling behavior, security operations, updates, monitoring, and platform reliability.
Global Standard is one of the common deployment types for Foundry Models. Microsoft documentation explains that Global Standard deployments use Azure's global infrastructure to route traffic dynamically to available datacenters, which helps support availability and inference capacity for standard usage patterns.
So from the developer's point of view, the deployed model becomes an endpoint. That endpoint is what our clients or applications call over HTTPS. A client could be a web application, a mobile app, a backend service, an internal business tool, or even another AI agent.
The client sends a request to the endpoint. That request normally includes the model deployment name, the user message or prompt, and optional parameters. The model processes the request and returns a response. The application then decides what to do with that response — show it in a chat interface, summarize a support ticket, generate an email, classify a document, or pass it into another workflow.
So the end-to-end flow is simple:
You deploy the model in Microsoft Foundry.
Microsoft runs and manages the infrastructure behind that deployment.
Your applications call the endpoint, send prompts, and receive model responses.
This is why model deployment is such a key foundation for AI application development. Before we build agents, RAG solutions,

workflows, or intelligent apps, we first need a reliable deployed model that our solution can call.

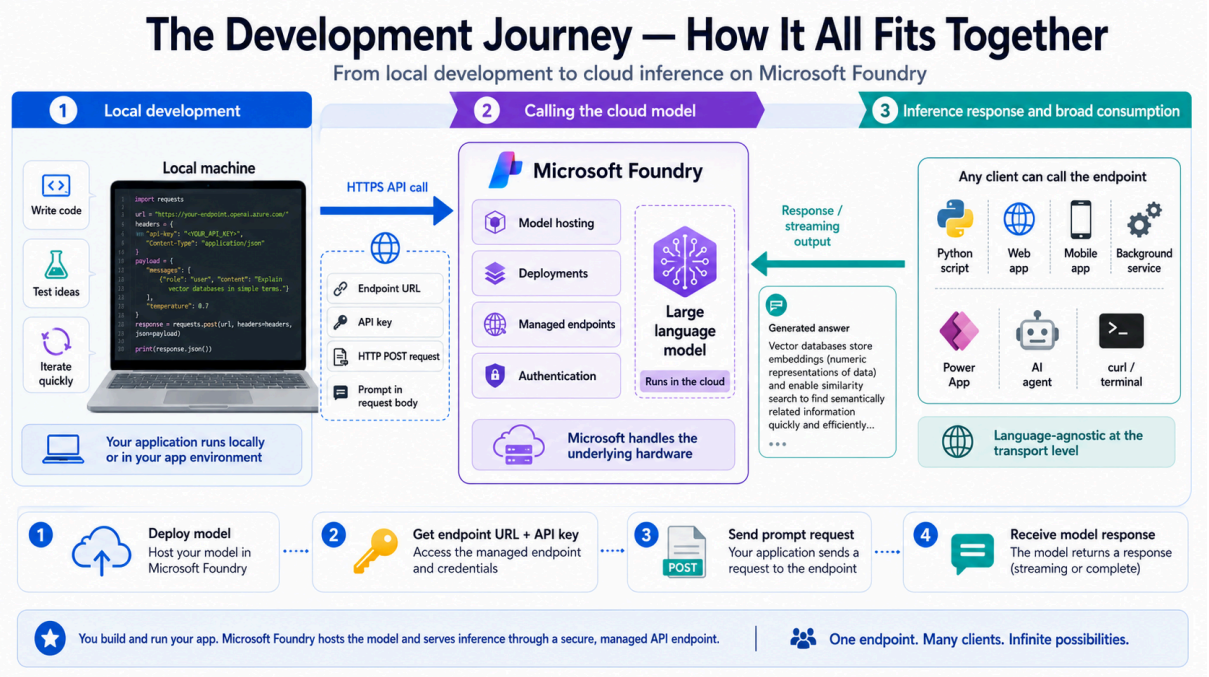
Review on model deployment





In this summary, we are looking at what happens when a model, such as GPT 5.4, is deployed in Microsoft Foundry.
The first step is to create a model deployment. In Microsoft Foundry, the model catalog lets us choose the model we want to use, configure the deployment, and make it available through an endpoint. This is an important distinction: selecting a model from the catalog is not enough. The model becomes usable by applications only after it has been deployed.
Once the deployment is created, Microsoft manages the underlying infrastructure. That means we do not need to provision virtual machines, install model runtimes, manage GPUs, or handle the hosting environment ourselves. With a deployment option such as Global Standard, Microsoft takes care of availability, scaling, security, monitoring, patching, and operational management.
After the model is deployed, client applications can call the model endpoint over HTTPS. These clients could be a web application, mobile app, backend service, developer workstation, or any other system that needs generative AI capabilities. The application sends a prompt as a request, and the deployed model returns a response.
This is the end-to-end flow students should remember: first deploy the model, then use the managed endpoint, and finally allow applications to send prompts and receive intelligent responses.
In a real business scenario, this is what allows an organization to add AI features into its applications without managing the model infrastructure directly. For example, a company could build a customer support assistant, invoice analysis tool, internal knowledge chatbot, or reporting assistant by calling the deployed model from its own application code.

Getting Started Python Development with Microsoft Foundry


The Development Journey — How It All Fits Together
From local development to cloud inference on Microsoft Foundry
1 Local development
2 Calling the cloud model
3 Inference response and broad consumption
Local machine
Write code
Test ideas
Iterate quickly
Your application runs locally or in your app environment
HTTPS API call
Endpoint URL
API key
HTTP POST request
Prompt in request body
Microsoft Foundry
Model hosting
Deployments
Managed endpoints
Authentication
Large language model
Runs in the cloud
Microsoft handles the underlying hardware
Response / streaming output
Generated answer
Vector databases store embeddings (numeric representations of data) and enable similarity search to find semantically related information quickly and efficiently...
Any client can call the endpoint
Python script
Web app
Mobile app
Background service
Power App
AI agent
curl / terminal
Language-agnostic at the transport level
1 Deploy model
Host your model in Microsoft Foundry
2 Get endpoint URL + API key
Access the managed endpoint and credentials
3 Send prompt request
Your application sends a request to the endpoint
POST
4 Receive model response
The model returns a response (streaming or complete)
You build and run your app. Microsoft Foundry hosts the model and serves inference through a secure, managed API endpoint.
One endpoint. Many clients. Infinite possibilities.



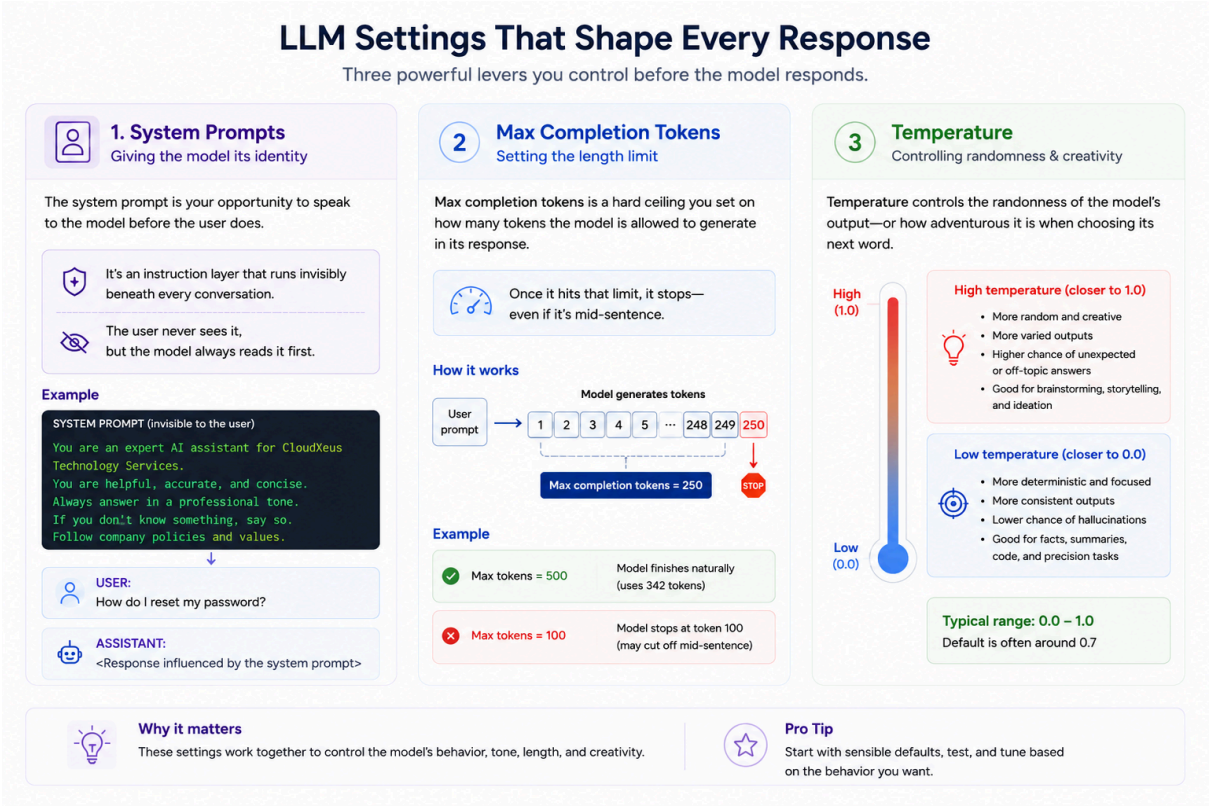
In this section, we are looking at the full development journey — how a Python application running on your machine can call a model that is deployed in Microsoft Foundry.
This is where we write and test our Python code on our own machine, usually in an editor like Visual Studio Code. At this stage, our application is still running locally, but it is not running the model locally. The model itself is hosted in Microsoft Foundry. Our code is simply preparing the request and sending it to the cloud endpoint.
Once we deploy a model in Foundry, we get a managed endpoint. That endpoint is the HTTPS address our application can call. Along with the endpoint, we also need authentication, such as an API key or another supported credential. These details allow our Python program to securely connect to the deployed model.
This is where the API request becomes important. The Python program sends a request to the endpoint. The request normally includes the prompt, the deployment name or model reference, and any settings we want to control, such as temperature, max output tokens, or reasoning effort. The model receives the request, processes it in the cloud, and generates a response.
Once the model generates the answer, the response comes back to our application. In a simple lab, we may just print the response in the terminal. But in a real solution, that same response could be displayed in a web app, used in a mobile app, passed to a Power App, processed by a background service, or used by an AI agent.
This is an important point for students to remember: the endpoint is language-agnostic at the transport level. That means Python is just one option. Any client that can make an authenticated HTTPS request can call the model endpoint. The same deployed model could be used by many different applications.

Lab - First API Call to a Foundry-Deployed Model



This first Python program shows the basic flow for calling a deployed model from code. We create an OpenAI client using the Foundry model endpoint and API key, then call `responses.create()` to send a prompt to the deployed model. The model processes the prompt and returns a response. Finally, `response.output_text` gives us the generated answer as plain text so we can print it or use it in our application.

Controlling Model Behaviour


LLM Settings That Shape Every Response
Three powerful levers you control before the model responds.
1. System Prompts
Giving the model its identity
The system prompt is your opportunity to speak to the model before the user does.
It's an instruction layer that runs invisibly beneath every conversation.
The user never sees it, but the model always reads it first.
Example
SYSTEM PROMPT (invisible to the user)
You are an expert AI assistant for CloudXeus Technology Services.
You are helpful, accurate, and concise.
Always answer in a professional tone.
If you don't know something, say so.
Follow company policies and values.
USER:
How do I reset my password?
ASSISTANT:
<Response influenced by the system prompt>
2
Max Completion Tokens
Setting the length limit
Max completion tokens is a hard ceiling you set on how many tokens the model is allowed to generate in its response.
Once it hits that limit, it stops—even if it's mid-sentence.
How it works
Model generates tokens
User prompt
1 2 3 4 5 ··· 248 249 250
STOP
Max completion tokens = 250
Example
Max tokens = 500
Model finishes naturally (uses 342 tokens)
Max tokens = 100
Model stops at token 100 (may cut off mid-sentence)
3
Temperature
Controlling randomness & creativity
Temperature controls the randonness of the model's output—or how adventurous it is when choosing its next word.
High (1.0)
Low (0.0)
High temperature (closer to 1.0)
More random and creative
More varied outputs
Higher chance of unexpected or off-topic answers
Good for brainstorming, storytelling, and ideation
Low temperature (closer to 0.0)
More deterministic and focused
More consistent outputs
Lower chance of hallucinations
Good for facts, summaries, code, and precision tasks
Typical range: 0.0 – 1.0
Default is often around 0.7
Why it matters
These settings work together to control the model's behavior, tone, length, and creativity.
Pro Tip
Start with sensible defaults, test, and tune based on the behavior you want.



In this section, we are looking at three settings that strongly influence how an LLM responds: system prompts, max completion tokens, and temperature.

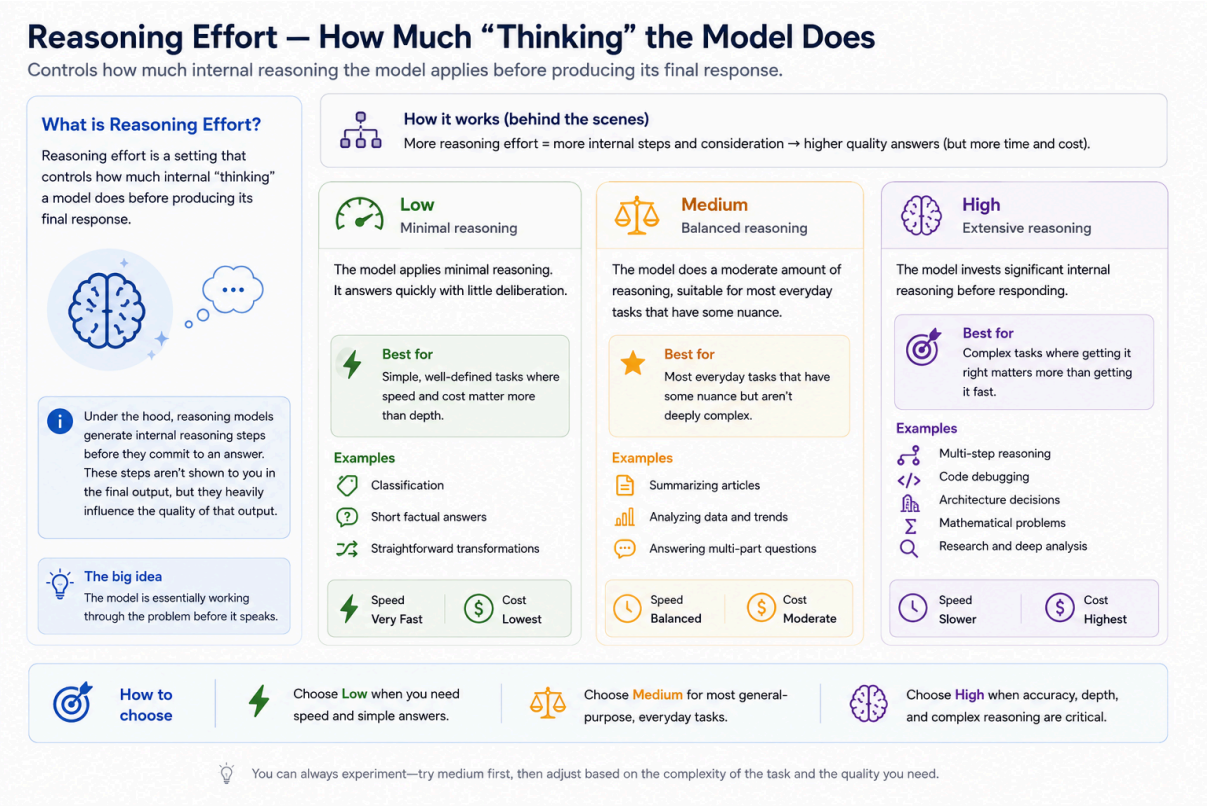
The first setting is the system prompt. This is where we define the model's identity and behavior before the user even asks a question. For example, we can tell the model that it is a support assistant for CloudXeus Technology Services, that it should answer in a professional tone, and that it should only use approved company guidance. The important point is that the system prompt is not normally shown to the user, but the model still follows it behind the scenes. This is how we give the model its role, tone, boundaries, and business rules.

The second setting is max completion tokens. This controls the maximum length of the model's generated response. The model generates its answer token by token, and this setting places a hard limit on how many output tokens it can produce. If the limit is high enough, the model can finish the response naturally. But if the limit is too low, the response may be cut off before the answer is complete. This setting matters because it helps control response length, cost, and performance.

The third setting is temperature. Temperature controls how predictable or creative the model's response should be. A lower temperature makes the model more focused and consistent, which is useful for factual answers, summaries, code explanations, and business support scenarios. A higher temperature allows more variation and creativity, which can be useful for brainstorming, marketing content, storytelling, or idea generation. So we do not choose temperature randomly. We choose it based on the type of task we want the model to perform.

The key takeaway is that these settings work together. The system prompt tells the model how to behave. Max completion tokens controls how long the response can be. Temperature controls how creative or predictable the response should be. By tuning these settings carefully, we can make the model's output more useful, more reliable, and better aligned with the business scenario.

Reasoning Effort Thinking Before Answering





Reasoning effort is important because not every task needs the same level of thinking. Some tasks are simple and direct. Others require analysis, planning, comparison, or step-by-step problem solving. This setting helps us decide how much reasoning the model should apply before producing its response.

The first option is low reasoning effort. This is useful when the task is simple, well-defined, and does not require deep analysis. For example, classification, short factual answers, or straightforward text transformations can usually work well with low reasoning effort. The benefit is speed and lower cost. The trade-off is that the model may not spend much time carefully considering complex details.

The second option is medium reasoning effort. This is a balanced setting and is often a good starting point for everyday use. It gives the model enough room to think through the request without adding too much extra time or cost. This is useful for tasks such as summarizing articles, analyzing trends, answering multi-part questions, or explaining concepts to users. For many business scenarios, medium reasoning provides a practical balance between quality, speed, and cost.

The third option is high reasoning effort. This is useful when accuracy, depth, and careful reasoning matter more than speed. For example, if the model needs to solve a complex coding problem, make an architecture recommendation, analyze research, or work through a multi-step decision, high reasoning effort can improve the quality of the answer. The trade-off is that it may take longer and cost more.

So the key point is that reasoning effort should match the complexity of the task. We do not always need the highest setting. For simple tasks, low reasoning may be enough. For general-purpose tasks, medium is usually a safe choice. For complex tasks where the answer really matters, high reasoning gives the model more room to think before responding.

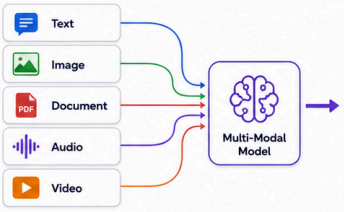
Lab - Multi-modal models

Multi-Modal Models

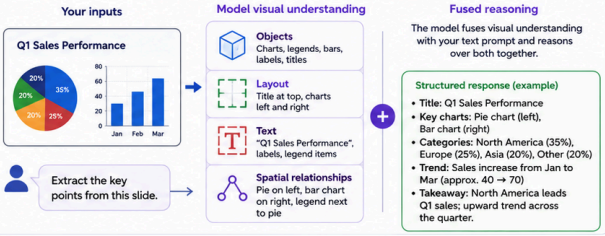
How modern AI works with text, images, audio, video, and documents together

1 What multi-modal means

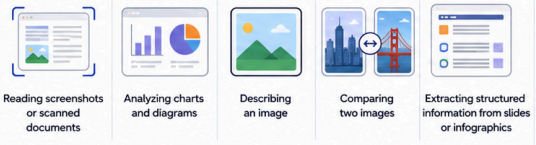
A multi-modal model can process more than one mode of data — most commonly text and images, and increasingly audio, video, and documents.



2 How it works



3 Practical use cases



4 Using it in the Responses API




```
JSON
input: [
  {type: "input_text", text: "Summarize this slide"},
  {type: "input_image", image_url: "https://...."}
]
```

You can send the image as a URL or base64-encoded content. The model treats the image as part of the conversation alongside your text prompt.

Multi-modal AI lets you ask natural-language questions about visual content and get useful, structured answers.



A multi-modal model is an AI model that can work with more than one type of input. Instead of only accepting text, it can also understand images, documents, audio, video, screenshots, charts, diagrams, and other visual content.

This is important because real business information does not always come as clean text. A customer may upload a screenshot. A finance team may work with scanned invoices. A support team may receive images of error messages. A manager may want to understand a chart in a report. Multi-modal models allow us to ask natural-language questions about these different types of content.

Text, images, documents, audio, and video can all become inputs to the model, depending on what the model supports. The model can then combine that information with the user's prompt to generate a useful response.

The user might provide an image of a chart and ask, "Extract the key points from this slide." The model does not just read the words. It can also understand objects, layout, labels, chart titles, colors, relationships, and visual structure. It then combines that visual understanding with language reasoning to produce a response.

This is where multi-modal reasoning becomes powerful. The model can look at a chart, identify the title, understand the bars or segments, compare values, and summarize the trend. Instead of only saying what text appears in the image, it can explain what the visual content means.

From a developer point of view, we build an input that can include both text and an image. The text tells the model what we want it to do, and the image provides the visual context. The model processes both together and returns a response, which can be plain text or structured output depending on how we design the request.

The key takeaway is that multi-modal AI lets users ask questions about visual content in the same natural way they ask questions about text. This helps us build richer AI applications that can understand documents, screenshots, images, charts, and business visuals — not just typed prompts.